

CAIRN: DYNAMIC FACT–INTENT DAGS FOR MULTI-AGENT EXPLORATION

Zuyao Xu^{†||} Yuyang Jia^{‡||} Junwei Guan[†] Xiang Li^{†*} Kaiwen Shen[§] Zhiqiang Dong[‡]

[†] Nankai University [‡] Tencent Security Yunding Lab [§] Tsinghua University

xuzuyao@mail.nankai.edu.cn lixiang@nankai.edu.cn

ABSTRACT

LLM-powered autonomous systems have demonstrated promising capabilities in mathematical reasoning, engineering, and cybersecurity. Yet how to organize these systems for effective, reliable, and sustained performance remains an open question. In this paper, we present CAIRN, a FACT–INTENT-driven multi-agent paradigm for goal-directed exploration. CAIRN represents observations and planned investigations as a dynamic directed acyclic graph (DAG). A REASONER interprets FACTs to propose INTENTS, which WORKERS execute to produce new FACTs. Each INTENT references its supporting FACTs and defines a potential exploration branch. The persistent graph preserves goals, dependencies and findings across WORKERS, supporting knowledge reuse and parallel exploration. The graph also makes execution trajectories traceable and auditable, providing a basis for human verification and intervention. We evaluate CAIRN across cybersecurity and mathematical reasoning tasks, examining task success, time to solution, and token consumption. DAG-based coordination can incur higher token costs with no observable performance gains on tasks that require little effort. However, on high-effort tasks ($\geq 1\text{M}$ tokens), we observe faster solutions in 76.5% of cases, with speedups of up to $3.08\times$. Moreover, as task effort increases, these time gains become more pronounced while relative token overhead declines, highlighting the potential of DAG-guided parallel exploration.

1 INTRODUCTION

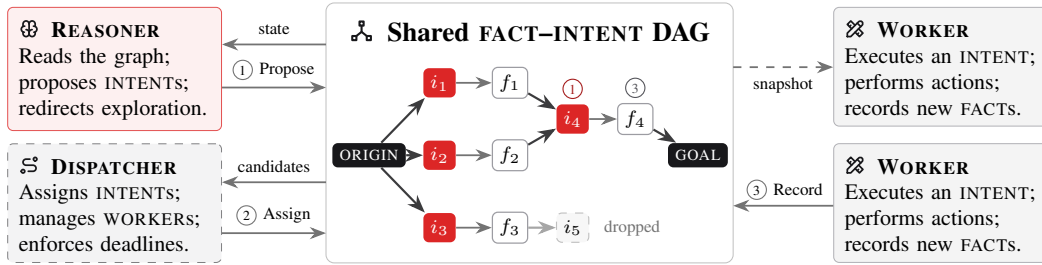


Figure 1: Exploration over a shared FACT–INTENT DAG. The REASONER reads graph state and proposes INTENTS citing source FACTs (1 Propose). The dashed DISPATCHER receives candidate INTENTS and assigns eligible candidates (2 Assign). WORKERS execute assigned INTENTS using a graph snapshot and record outcome FACTs (3 Record). In the illustrated graph, f_1 and f_2 support i_4 . The REASONER drops i_5 when new evidence contradicts its underlying premise.

Many difficult intellectual tasks require extended effort and collaboration for humans to solve, yet recent successes of large language models (LLMs) suggest that multi-agent systems can achieve similar results in a fraction of the time (OpenAI, 2026c; Anthropic, 2026a). For example, coordinated

*Corresponding author

^{||}Equal Contribution

agents have contributed to research on two Millennium Prize Problems, with a reported breakthrough on the Navier–Stokes problem and new results related to the Riemann hypothesis (OpenAI, 2026c; Anthropic, 2026b). Project Glasswing and OpenAI’s Daybreak likewise incorporate multi-agent workflows into security efforts, with Glasswing reporting over 10,000 high- or critical-severity vulnerabilities found in one month and Daybreak reporting 143 upstream-accepted patches (Anthropic, 2026a; OpenAI, 2026a;b). Together, these advances point to a new level of capability in harnessing LLMs through multi-agent collaboration.

Despite these successes, a central question remains: *how to coordinate multiple agents to collaborate effectively and sustainably?* Prior work offers several approaches. MetaGPT and ChatDev organize software-development agents through specialized roles and workflows (Hong et al., 2024; Qian et al., 2024). AutoGen provides a programmable framework for multi-agent conversation (Wu et al., 2024). More recent frameworks dynamically create task-specific sub-agents (Ruan et al., 2026) and coordinate dependent subtasks through verification-aware plans (Xu et al., 2026).

In this paper, we focus on *heuristic problem solving*: tasks with a clear objective and an open solution path. Solving such tasks may require pursuing multiple possible directions over time. Candidate approaches may be proposed, tested, refined, or abandoned as understanding develops. Motivated by such problems, we introduce CAIRN, a multi-agent coordination paradigm that represents findings as FACTS and planned investigations as INTENTS. A REASONER proposes INTENTS that explicitly reference their supporting FACTS, and WORKERS execute these INTENTS, contributing new FACTS to the graph (Figure 1). These dependencies form a persistent directed acyclic graph: one FACT can motivate several INTENTS, and several FACTS can jointly motivate a new INTENT. The graph preserves these connections across WORKERS and sessions, helping agents reuse findings and coordinate parallel exploration. It also makes each investigation traceable, showing what motivated it, what it found, and how findings from different branches informed later directions.

We evaluate CAIRN on twelve public mathematical reasoning problems from FrontierMath (Epoch AI, 2026) and forty cybersecurity tasks from Cybench (Zhang et al., 2025), comparing it with a baseline agent using the same model, agent loop, and task environment. On high-effort tasks ($\geq 1\text{M}$ baseline tokens), CAIRN finishes sooner in 76.5% of cases, with speedups of up to $3.08\times$. Although CAIRN uses more tokens on average, its relative token overhead decreases as task effort increases. We also manually reviewed sixteen FACT–INTENT DAGs from FrontierMath runs, finding that they reveal how investigations reuse earlier findings, combine evidence, and correct errors. These graphs provide structured records for human review, verification, and auditing of multi-agent problem solving. Together, our results suggest a role for coordinated exploration in sustained tasks where faster completion justifies additional computation, including hyperscale deployments with abundant inference capacity. In practice, an open-source implementation of this paradigm has attracted approximately 3,000 GitHub stars (oriterra, 2026). A system built on the paradigm was the only one to solve all tasks in the second Tencent Cloud Intelligent Penetration Hackathon (Tencent Cloud Yunding Lab, 2026) and achieved a leading result in the public TSecBench cybersecurity evaluation (TSecBench, 2026).

2 CAIRN: EXPLORATION THROUGH FACT–INTENT DAGS

Figure 2 presents two approaches to the same credential-recovery task: a sequential action–observation trajectory in panel (a) and CAIRN’s FACT–INTENT DAG in panel (b). In panel (b), CAIRN represents the task through an ORIGIN, which records the initial information, and a GOAL, which specifies the desired outcome. CAIRN records planned investigations as INTENTS and their findings as FACTS, linking each INTENT to the FACTS that motivate it and the findings it produces. Task solving is thus represented as the incremental construction of a persistent directed acyclic graph (DAG) from the ORIGIN toward the GOAL.

2.1 TASK SOLVING AS GRAPH CONSTRUCTION

CAIRN constructs the task graph as it investigates possible routes from the ORIGIN toward the GOAL. Proposing an investigation adds an INTENT linked to one or more existing FACTS that motivate it. As the investigation proceeds, its findings are added as new FACTS linked to that INTENT. These findings can motivate further investigations, extending the graph through successive FACT–INTENT–

Illustrative task: Investigate a host exposing a web portal and an admin API protected by session tokens. The portal stores its data in a backend database. Recover the credentials needed to access that database.

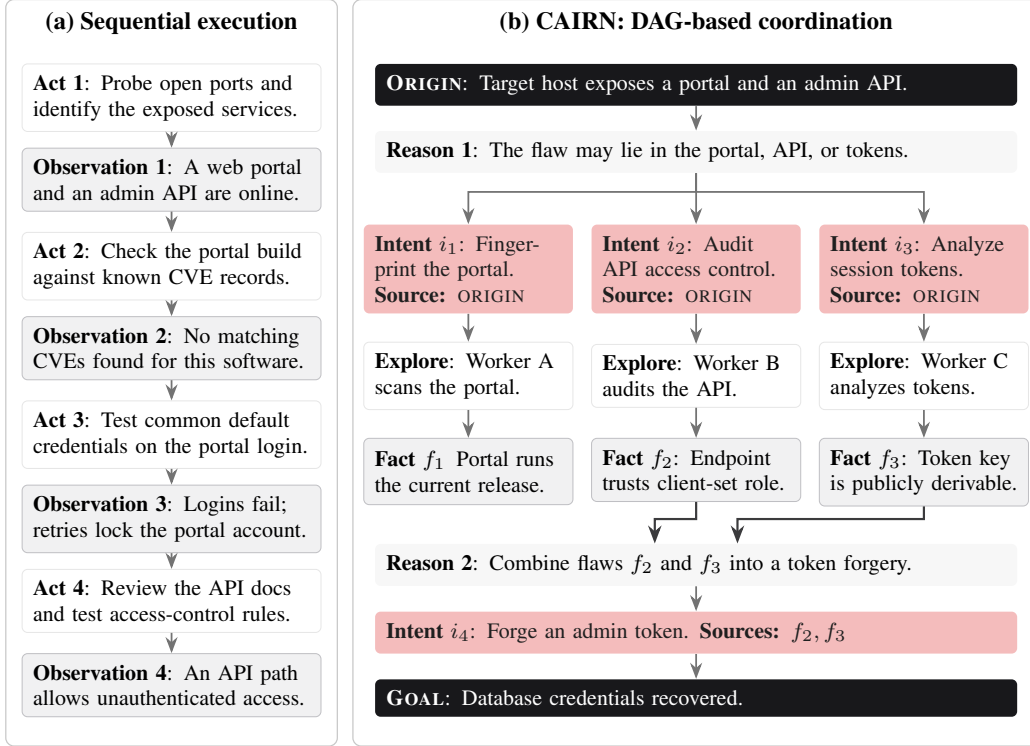


Figure 2: Two approaches to the same credential-recovery task. (a) A sequential action–observation trajectory. (b) CAIRN’s FACT–INTENT DAG. Three investigations start from the ORIGIN and examine the portal, API, and session tokens. The API and token findings jointly motivate a subsequent investigation toward the GOAL.

FACT steps. When the accumulated evidence is judged to satisfy the task objective, a completion INTENT connects the supporting FACTS to the terminal GOAL.

Starting with the ORIGIN and GOAL nodes, exploration extends the graph through three operations:

1. *Propose*: add an INTENT citing the FACTS that motivate it.
2. *Assign*: dispatch an eligible INTENT to a WORKER for investigation.
3. *Record*: publish findings as new FACTS that can support later INTENTS.

The graph supports collaboration through *branching* and *convergence*. Shared FACTS can motivate separate INTENTS, allowing agents to pursue different directions; findings from these branches can then jointly support a subsequent INTENT. A formal definition of the graph and its operations is provided in Appendix A.

2.2 COORDINATION THROUGH A SHARED GRAPH

We implement CAIRN with two agent roles: a REASONER and a set of WORKERS (Figure 3). The DAG serves as a persistent shared memory, making recorded evidence and investigation state available across agents. The REASONER uses this memory to propose INTENTS and redirect investigations, while WORKERS carry out assigned investigations using task tools and publish findings as FACTS. A rule-based DISPATCHER initiates reasoning and assigns eligible INTENTS to WORKERS. Each invocation of either agent role constitutes a *session*, initialized with a graph snapshot and role-specific context. Both roles submit updates through a graph service that validates and persists accepted changes.

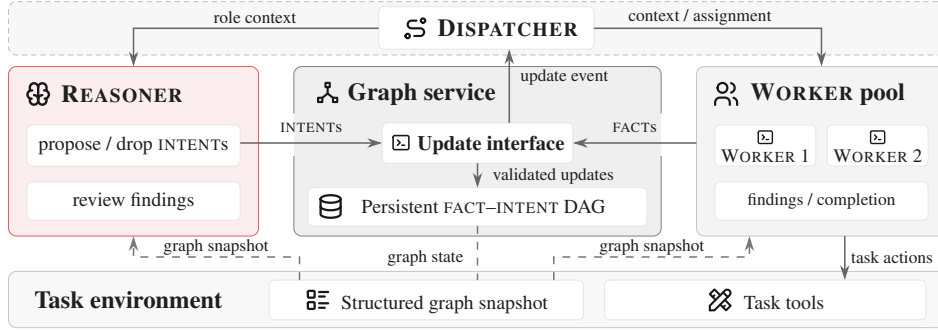


Figure 3: CAIRN execution architecture. The DISPATCHER starts REASONER and WORKER sessions with graph snapshots. SESSIONS submit changes through the graph service; WORKERS also use task tools. Accepted updates persist for later sessions and notify the DISPATCHER. Solid arrows show assignments, updates, and task actions; dashed links show graph state and snapshots.

Independent investigations can proceed in parallel. Once accepted, a finding is available to later sessions while its producing WORKER continues the investigation. The REASONER can use these findings to propose follow-up investigations, allowing branches to develop alongside ongoing work.

2.3 INSPECTABLE TRAJECTORIES AND REVISION

The graph’s source and output edges expose an inspectable investigation trajectory: which findings motivated a direction, what it produced, and where branches split or join. The graph traces investigations and their relationships at a higher level than individual individual agent actions. A FACT records an agent-reported finding whose reliability may change as evidence accumulates; it may describe a failed attempt, a provisional observation, or a claim later contradicted by new evidence.

For robustness, the REASONER have the ability to check and drop INTENTS, stopping further updates from any assigned WORKERS, allowing it to correct earlier mistakes or redirect investigations,. A WORKER can publish corrective evidence as a new FACT, which replacement INTENTS can cite to extend the recorded trajectory with new branches. The graph preserves the investigation history, including the records of dropped INTENTS, allowing later reviewers to understand the reasoning behind the revision. We present a detailed description of CAIRN’s implementation in Appendix C.

3 EXPERIMENT SETUP

In this section, we describe the experimental setup for evaluating CAIRN. We summarize four factors shaping agent behavior in Figure 4. The *model* affects reasoning capabilities and learned knowledge, while the *agent loop* shapes tool interaction and context management. The *task architecture* governs how work is organized and how information is retained and shared. The *execution environment* determines the runtime facilities, task tools, and access permissions during action.

Among these factors, CAIRN is positioned at the task-architecture level, coordinating multi-agent exploration through a shared FACT-INTENT graph. To provide a transparent basis for assessing its effect on task solving, we compare CAIRN with a direct agent baseline (referred to as DIRECT) using the same model, agent-loop implementation, and task environment, treating the high-level coordination paradigm as the variable of interest.

Benchmarks. We choose two benchmarks with externally checkable outcomes and a range of task durations, both used in prior work (Udeshi et al., 2025; Kouremetis et al., 2026; Zheng et al., 2026; McFadyen et al., 2026):

1. *FrontierMath* is a mathematical reasoning benchmark with problems of increasing difficulty from Tier 1 to Tier 4. According to Epoch AI, Tier 3 and Tier 4 problems can occupy specialists for days; leading mathematicians have described them as “exceptionally challenging” (Epoch AI, 2026). We use the twelve publicly available sample problems.

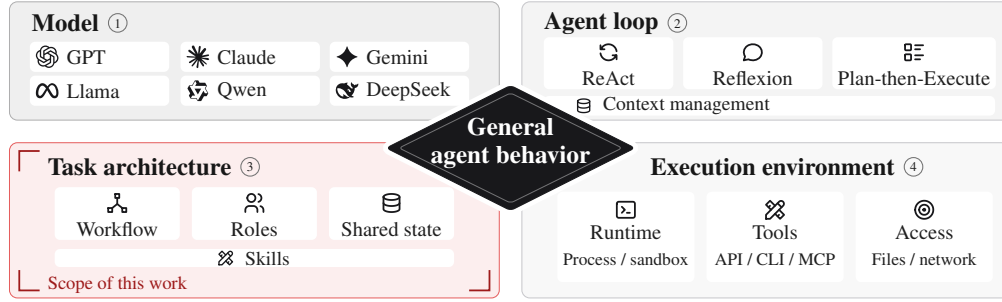


Figure 4: General agent behavior design factors. CAIRN focuses on task architecture as a multi-agent coordination paradigm, while other factors are held constant as the basis for comparison in the evaluation. The examples in each area illustrate representative design choices.

2. *Cybench* is a cybersecurity benchmark that provides CTF tasks with interactive environments and externally checkable outcomes (Zhang et al., 2025). We use the unguided complete-task setting: solvers receive the original task, permitted files, and target access without benchmark subtask hints.

Systems setup. We use Pi 0.85.1 (Earendil Inc., 2025) as our minimal agent implementation and GLM-5.1 (Z.ai, 2026) as the shared model. DIRECT retains Pi’s native system prompt, local tools, and context management; CAIRN replaces the prompt with role-specific instructions and adds graph-based collaboration tools, with up to six execution slots shared by the REASONER and WORKERS. Both systems run in isolated containers built from the same benchmark-specific image, with public Internet access disabled. Within a five-hour hard limit, agents may resubmit after feedback from external verifiers. We report solving time to the first accepted answer, assigning five hours to runs that reach the limit unsolved, and total token usage per run. Configuration, environment, and measurement details appear in Appendix D.

4 RESULTS

In this section, we present the results of our evaluation of CAIRN on two benchmarks: FrontierMath (Epoch AI, 2026) and Cybench (Zhang et al., 2025).

4.1 BENCHMARK-LEVEL RESULTS

Table 1 summarizes solving time and token usage on each benchmark. On FrontierMath, CAIRN has a lower mean solving time than DIRECT (27.45 versus 53.19 minutes) and finishes sooner on 8 of 12 problems (66.7%), including all five Tier-3 and Tier-4 problems. This time advantage comes with higher mean total token usage: 13.44M for CAIRN versus 2.81M for DIRECT. On Cybench, DIRECT finishes sooner on 30 of 40 tasks (75%), with lower mean solving time (7.31 versus 8.49 minutes) and token usage (0.83M versus 3.22M). The contrast between benchmarks motivates us to examine the results further at the task level.

4.2 TASK-DEPENDENT PERFORMANCE

Figure 5 compares the two systems jointly in terms of solving time and total token usage. Across the 52 tasks, CAIRN has faster solving time on 18. On the remaining 34 tasks, CAIRN uses both more time and more tokens. At the task level, we observe that CAIRN tends to achieve a greater solving-time advantage on tasks with higher token usage under DIRECT. We therefore use DIRECT’s total token usage, averaged across the runs reported for each task, as an empirical measure of *task effort* under the common model and agent loop. Table 2 summarizes performance across three effort groups, pooling tasks from both benchmarks. In the highest-effort group ($\geq 1\text{M}$ baseline tokens), CAIRN reduces mean solving time from 47.20 to 25.35 minutes, whereas DIRECT is faster on average in the two lower-effort groups. The mean per-task token ratio (CAIRN/DIRECT) decreases from 14.71 in the lowest-effort group to 3.69 in the highest-effort group.

Table 1: Benchmark-level summary of mean solving time and token usage on FrontierMath and Cybench.

Metric	DIRECT	CAIRN
<i>FrontierMath</i>		
Runs	36	36
Faster	4/12	8/12
Time (min)	53.19	27.45
Tokens (M)	2.81	13.44
<i>Cybench</i>		
Runs ^a	40	40
Faster ^b	30/40	10/40
Time (min)	7.31	8.49
Tokens (M)	0.83	3.22

^a Runs: evaluation runs.

^b Faster: paired tasks solved sooner.

Table 2: Time–compute profile by task effort. Effort is DIRECT’s mean token usage for solving a task. For each group, we report run counts, mean solving time and token usage, and mean per-task ratios for DIRECT and CAIRN.

Effort (M), n	Sys. ^a	Runs	Time (min)	Tokens (M)	Mean ratio ^b	
					Time $\uparrow$	Tokens $\downarrow$
≥ 1 (17)	D	29	47.20	3.48	1.67	3.69
	C	29	25.35	13.56		
[0.1, 1) (20)	D	30	5.43	0.35	0.66	8.66
	C	30	9.13	2.54		
< 0.1 (15)	D	17	1.31	0.04	0.42	14.71
	C	17	3.72	0.60		
All (52)	D	76	17.90	1.28	0.92	8.78
	C	76	12.87	5.58		

^a Sys. = system; D = direct agent; C = CAIRN; n = tasks.

^b Time ratio: D/C; token ratio: C/D. $\uparrow/\downarrow$: higher/lower better.

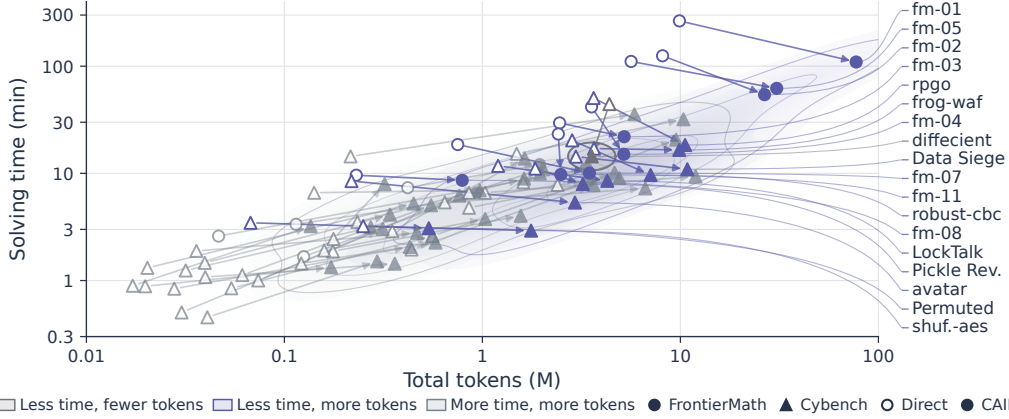


Figure 5: Time–token trade-offs across the 52 tasks. Arrows run from DIRECT to CAIRN; labels on the right name the tasks on which CAIRN finishes sooner than DIRECT; lines connect each label to its CAIRN endpoint. Shading shows where CAIRN endpoints cluster within each outcome group.

We further illustrate this pattern in Figure 6, which shows per-task ratios and their distributions within the same three effort groups. Across these groups, the mean solving-time ratio (DIRECT/CAIRN) rises with task effort, exceeding 1 in the high-effort group, while the mean token ratio (CAIRN/DIRECT) falls.

Interpreting benchmark-level differences. The distribution of task effort helps explain the contrasting benchmark averages; the bottom row of Figure 6 shows their task-level patterns. 29 of the 40 Cybench tasks (72.5%) fall in the low- and mid-effort groups (< 1 M baseline tokens), while 6 of the 12 FrontierMath problems (50%) fall in the high-effort group (≥ 1 M). Within Cybench, CAIRN finishes sooner on 7 of 11 (63.6%) high-effort tasks, compared with 3 of 29 (10.3%) low- and mid-effort tasks. On FrontierMath, CAIRN finishes sooner on all 6 (100%) high-effort problems, compared with 2 of 6 low- and mid-effort problems (33.3%). The greater proportion of high-effort problems in FrontierMath is consistent with its stronger aggregate time advantage.

We further examine the relationship between task effort and time advantage across both benchmarks in Figure 7. The continuous fit shows a positive association between baseline effort and CAIRN’s time advantage, with an estimated break-even point of 1.24M baseline tokens. These results characterize CAIRN’s task-dependent profile: as task effort increases, its solving-time advantage becomes more pronounced while its relative token overhead declines.

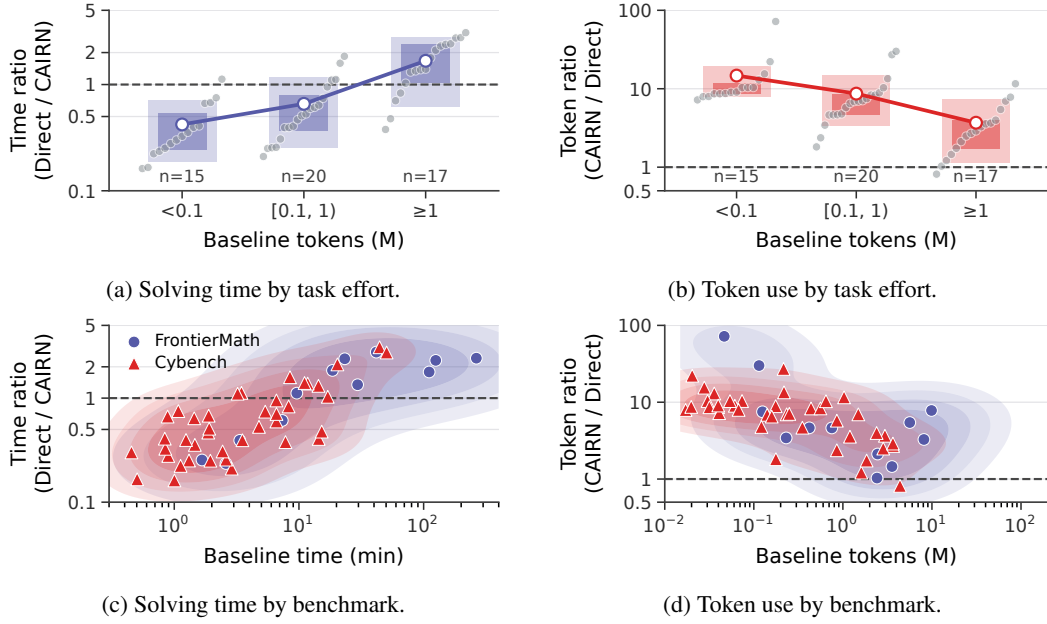


Figure 6: Task-level performance by effort and benchmark. In (a,b), points show tasks in the three baseline-token effort groups of Table 2; lines connect arithmetic means, light and dark boxes span the 10th–90th and 25th–75th percentiles, and n gives group size. In (c,d), circles denote FrontierMath and triangles Cybench tasks; shaded contours show task density.

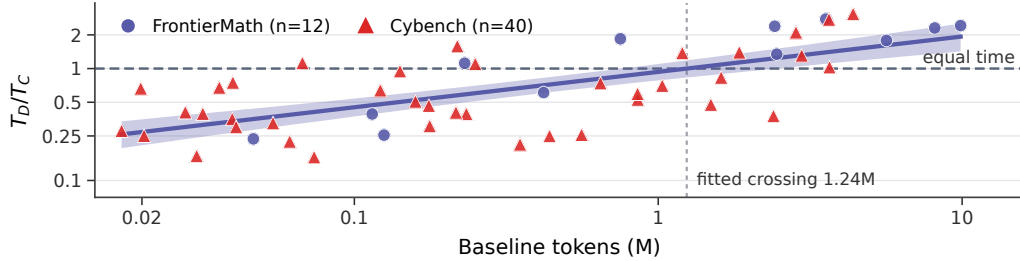


Figure 7: Time advantage versus task effort. The vertical axis shows T_D/T_C , where T_D is the solving time of DIRECT and T_C is CAIRN; values above one favor CAIRN. The line shows a pooled log-log linear fit with a pointwise 95% task-bootstrap confidence band. The horizontal line marks equal time ($T_D/T_C = 1$), and the vertical dotted line marks the fitted crossing at 1.24M tokens.

This pattern suggests that CAIRN may be particularly useful for complex and challenging tasks where solving time is critical and additional token usage is less of a concern, including hyperscale deployments by model providers with abundant inference capacity.

4.3 CONCURRENCY ANALYSIS

Next, we examine how the number of execution slots relates to solving time and token usage in CAIRN. We analyze active WORKERS in three successful runs of the Tier-4 FrontierMath problem fm-02, a high-effort task on which CAIRN shows a time advantage (Figure 8).

With one, three, and six execution slots, the recorded solving times are 97.22, 73.70, and 26.51 minutes, respectively, while the number of active WORKERS peaks at one, three, and five. Across these runs, greater concurrency accompanies shorter solving times, with the six-slot run completing substantially sooner than the other two. Furthermore, the corresponding token usage totals are

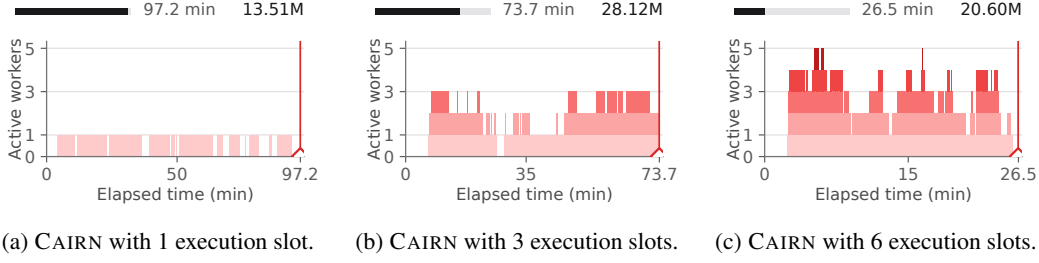


Figure 8: Active WORKERS in three CAIRN runs on the same FrontierMath problem (fm-02). Each execution slot runs one agent session at a time; REASONER and WORKER sessions share the available slots. Colored bands count simultaneously active WORKERS; blank intervals may include REASONER activity or tool execution.

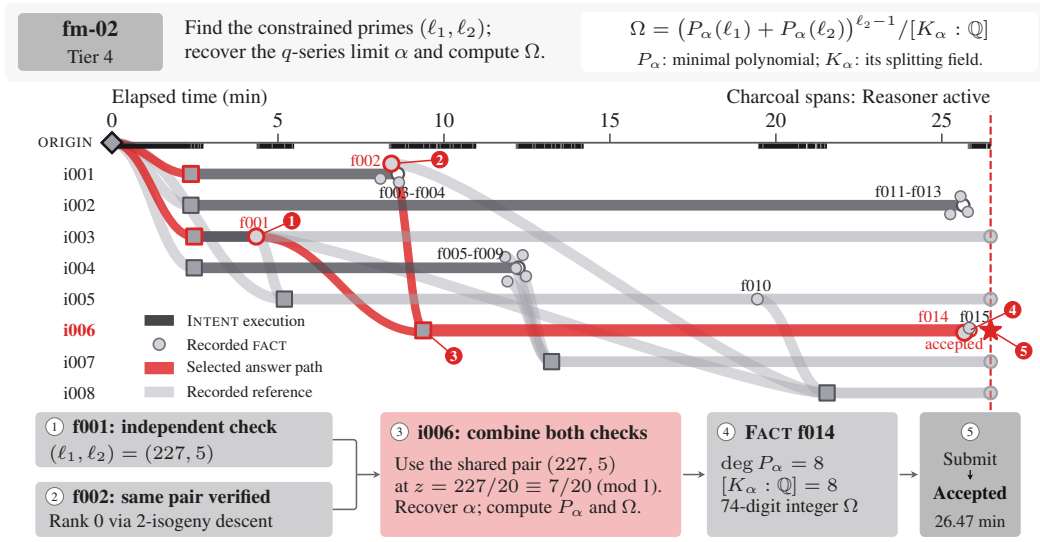


Figure 9: Execution timeline for fm-02. The 26.47-minute timestamp measures submission from the first model request; 26.51-minute solving time includes initialization and verification. Squares mark INTENT starts, circles mark FACTS, and charcoal spans mark REASONER activity. Pale streams lack recorded completion before submission.

13.51M, 28.12M, and 20.60M; the six-slot run finishes sooner than the three-slot run while using fewer recorded tokens, suggesting that greater concurrency may not result in greater token use.

These activity profiles illustrate how CAIRN supports parallel investigations during high-effort task solving, allowing WORKERS to explore multiple directions concurrently, providing a plausible explanation for the observed time advantage compared to single-agent execution.

4.4 EXECUTION TRACE ANALYSIS

CAIRN’s FACT–INTENT DAGs also provide a more desirable view of agent activity for audit and attribution, supporting auditable problem solving and agent traceability. We manually reviewed sixteen FrontierMath DAGs to examine how CAIRN organizes and coordinates concurrent investigations, records their findings, and proposes subsequent work. These traces reveal how the shared graph supports solution construction, error correction, and verification.

Figure 9 demonstrates the execution trace for a single CAIRN run on the Tier-4 FrontierMath problem fm-02, completed in 26.51 minutes. Following an initial planning phase, overlapping investigations publish f001 and f002, which report independent checks of a prime pair. INTENT i006 cites both findings to recover the target quantity, publishing f014 with the value used in the accepted

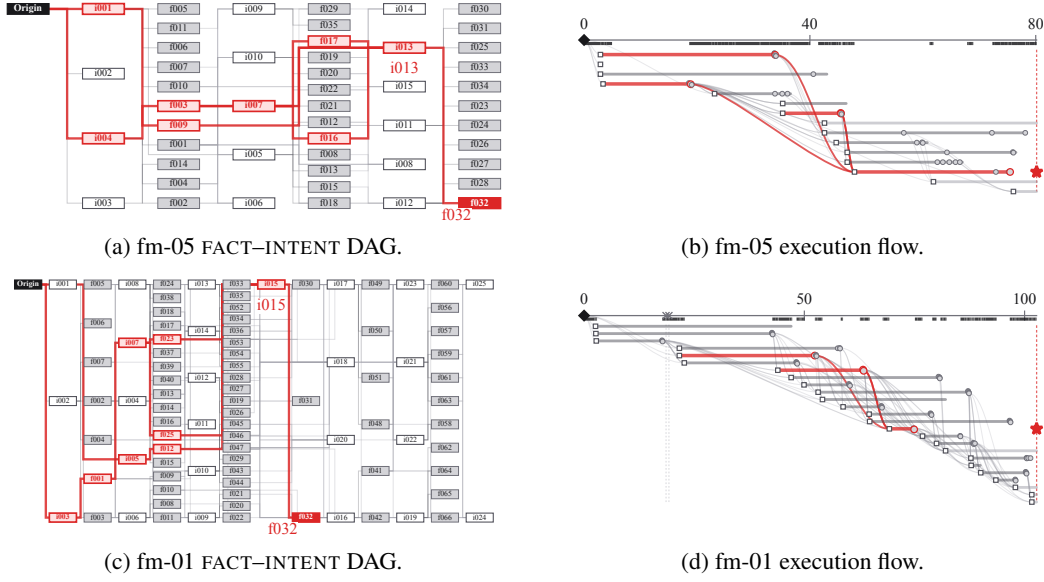


Figure 10: Complete coordination traces for two FrontierMath CAIRN runs: fm-05 (top) and fm-01 (bottom). Left: FACT-INTENT DAGs. Right: execution flows with elapsed time in minutes. Red highlights answer-related paths, and stars mark accepted submissions.

submission. These recorded links connect the parallel investigations to their convergence in the submitted solution, showing how CAIRN supports concurrent work and integrates its results.

Figure 10 pairs the archived DAGs with execution flows for two further FrontierMath runs, fm-05 and fm-01. The total token usage for these runs is 43.0M and 77.8M input-plus-output tokens, respectively, with archived request/response bodies of 130 and 256 MB. Without the DAGs, exhaustive human review of these raw records is impractical, making the agent trajectory opaque and difficult to audit. CAIRN’s DAG design addresses this challenge by providing a structured representation of the agent’s reasoning process. The DAGs expose which INTENTS produced and reused each FACT, while the execution flows show when those events occurred. These paired views make agent runtime trajectories more legible and support targeted audit and attribution.

5 RELATED WORK

Coordinated exploration raises two connected questions: how should agents organize their work, and how should findings guide later work? Prior systems have structured collaboration through role-based workflows and programmable conversations (Hong et al., 2024; Qian et al., 2024; Wu et al., 2024), while adaptive approaches create task-specific sub-agents or learn orchestration configurations (Ruan et al., 2026; Ke et al., 2026). Interactive security adds a concrete setting for these ideas, with systems maintaining task context, operating terminal tools, and feeding execution results back into planning (Deng et al., 2024; Abramovich et al., 2025; Udeshi et al., 2025).

The need to carry findings forward also motivates shared-state approaches. Blackboard architectures let independent components contribute to a common knowledge store (Erman et al., 1980), while graph-based methods connect reasoning states, evolve agent communication, or encode verification dependencies (Besta et al., 2024; Zhang et al., 2026; Xu et al., 2026). CAIRN links these strands in a shared FACT-INTENT DAG: recorded findings motivate executable investigations, whose outputs can guide later branches and leave an inspectable evidence trail.

6 CONCLUSION

In this paper, we present CAIRN, a paradigm for coordinating multi-agent exploration through a shared FACT-INTENT DAG. Our evaluation on FrontierMath and Cybench reveals a task-dependent

time–compute trade-off: CAIRN achieves greater time advantages on high-effort tasks with relative overhead decreasing as task effort increases. CAIRN offers a way to organize parallel investigations while keeping their findings and interactions inspectable.

AI USE STATEMENT

In accordance with the ICLR 2027 policy, we disclose the use of generative AI in our study. Generative AI was used to assist manuscript drafting and revision, and preparation of LaTeX and build artifacts. Language agents are the system under study, and their outputs are evaluated as part of the experimental results. All AI-generated outputs were reviewed and verified by the authors, who bear full responsibility for the scientific accuracy and integrity of the final manuscript.

ETHICS STATEMENT

We evaluate security agents only on authorized benchmark tasks in controlled environments. All agents operate in isolated containers with no access to the public Internet or any third-party systems. We do not use any private or sensitive data in our evaluation, and we do not deploy agents on live systems. All results are reproducible using publicly available benchmarks, open-source agent implementations, and the shared model.

REPRODUCIBILITY STATEMENT

Readers can follow the evaluation from CAIRN’s graph protocol and execution architecture (Section 2), through the experimental conditions (Section 3), to the reported results (Section 4). Appendices A–D then give the algorithms, role prompts, tool protocol, and detailed setup behind those runs. We will also submit anonymized source code and configuration files in the supplementary material, including the CAIRN implementation, evaluation scripts, and benchmark images to support independent reproduction of the reported results.

REFERENCES

- Talor Abramovich, Meet Udeshi, Minghao Shao, Kilian Lieret, Haoran Xi, Kimberly Milner, Sofija Jancheska, John Yang, Carlos E. Jimenez, Farshad Khorrami, Prashanth Krishnamurthy, Brendan Dolan-Gavitt, Muhammad Shafique, Karthik R. Narasimhan, Ramesh Karri, and Ofir Press. EnIGMA: Interactive tools substantially assist lm agents in finding security vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 246–355, 2025. URL <https://proceedings.mlr.press/v267/abramovich25a.html>.
- Anthropic. Project Glasswing: An initial update, 2026a. URL <https://www.anthropic.com/research/glasswing-initial-update>. Progress report, May 22, 2026.
- Anthropic. Learning more about Claude’s mathematical capabilities, 2026b. URL <https://www.anthropic.com/research/riemann-zeta>. Research article, August 10, 2026; updated August 13, 2026.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Giniak, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024. doi: 10.1609/aaai.v38i16.29720. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29720>.
- Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium*, pp. 847–864. USENIX Association, 2024. URL <https://www.usenix.org/conference/usenix-security24/presentation/deng>.

- Earendil Inc. Pi: An open-source agent toolkit and coding agent, 2025. URL <https://pi.dev>. Version 0.85.1.
- Epoch AI. FrontierMath: Benchmark problems, 2026. URL <https://epoch.ai/frontiermath/tiers-1-4/benchmark-problems>. Public sample problems; local input snapshot recorded September 10, 2026.
- Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2):213–253, 1980. doi: 10.1145/356810.356816. URL https://mas.cs.umass.edu/pub/paper_detail.php/229.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, pp. 23247–23275, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html.
- Zixuan Ke, Yifei Ming, Austin Xu, Ryan Chin, Xuan-Phi Nguyen, Prathyusha Jwalapuram, Jiayu Wang, Semih Yavuz, Caiming Xiong, and Shafiq Joty. Mas-Orchestra: Understanding and improving multi-agent reasoning through holistic orchestration and controlled benchmarks. In *International Conference on Machine Learning*, 2026. URL <https://icml.cc/virtual/2026/poster/66445>.
- Michael Kouremetis, Ads Dawson, Raja Sekhar Rao Dheekonda, and Brian Greunke. Every Model Cheats: Prompt-level mitigation of cheating on offensive cyber tasks. *arXiv preprint arXiv:2607.21763*, 2026. URL <https://arxiv.org/abs/2607.21763>.
- Jessica McFadyen, Ole Jorgensen, Harry Coppock, Kevin Wei, and Cozmin Ududec. How inference compute shapes frontier LLM evaluation. *arXiv preprint arXiv:2606.17930*, 2026. URL <https://arxiv.org/abs/2606.17930>.
- OpenAI. Daybreak: OpenAI for cybersecurity, 2026a. URL <https://openai.com/daybreak/>. Program results dashboard, accessed September 21, 2026.
- OpenAI. Codex Security: Deep security scan, 2026b. URL <https://github.com/openai/plugins/blob/main/plugins/codex-security/skills/deep-security-scan/SKILL.md>. Public workflow specification, accessed September 21, 2026.
- OpenAI. On the Navier–Stokes millennium prize problem, 2026c. URL <https://openai.com/index/navier-stokes-solution/>. Research report, September 8, 2026.
- oritera. Cairn: Open-source implementation, 2026. URL <https://github.com/oritera/Cairn>. GitHub repository; star count observed September 26, 2026.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.810. URL <https://aclanthology.org/2024.acl-long.810/>.
- Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. AOrchestra: Automating sub-agent creation for agentic orchestration. In *International Conference on Machine Learning*, 2026. URL <https://icml.cc/virtual/2026/poster/65930>.
- Tencent Cloud Yunding Lab. Official event resources and second-edition results, 2026. URL <https://github.com/Yeti-791/Tsec-Hackathon>. Official event resource repository; the second-edition project list identifies the Cairn-linked entry as the only team to complete all challenges.

- TSecBench. TSecBench: Public agent evaluation results and leaderboards, 2026. URL <https://tsecbench.zc.tencent.com/api/v1/comparison-reports>. Public comparison records, accessed September 26, 2026; platform homepage: <https://tsecbench.zc.tencent.com/>.
- Meet Udeshi, Minghao Shao, Haoran Xi, Nanda Rani, Kimberly Milner, Venkata Sai Charan Putrevu, Brendan Dolan-Gavitt, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad Shafique. D-CIPHER: Dynamic collaborative intelligent multi-agent system with planner and heterogeneous executors for offensive security. *arXiv preprint arXiv:2502.10931*, 2025. URL <https://arxiv.org/abs/2502.10931>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
- Tianyang Xu, Dan Zhang, Kushan Mitra, and Estevam Hruschka. Verification-aware planning for multi-agent systems. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7528–7546. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.eacl-long.353. URL <https://aclanthology.org/2026.eacl-long.353/>.
- Z.ai. GLM-5.1: Release notes, 2026. URL <https://docs.z.ai/release-notes/new-released>. Model release entry dated April 7, 2026.
- Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Haoxiang Yang, Aolin Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Kenny Oseleononmen, Dan Boneh, Daniel Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2408.08926>.
- Heng Zhang, Yihao Zhong, Lubin Gan, Zhihe Chen, Jiajun Wu, Yuling Shi, Xiaodong Gu, Hao Zhang, Haochen You, and Jin Huang. EvoHyper: Evolving hypergraph topologies for unified collaboration in multi-agent communication. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 25130–25143. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1258. URL <https://aclanthology.org/2026.findings-acl.1258/>.
- Daniel Zheng, Ingrid von Glehn, Yori Zwols, Iuliya Beloshapka, Lars Buesing, Daniel M. Roy, Martin Wattenberg, Bogdan Georgiev, Tatiana Schmidt, Andrew Cowie, Fernanda Viegas, Dimitri Kanevsky, Vineet Kahlon, Hartmut Maennel, Sophia Alj, George Holland, Alex Davies, and Pushmeet Kohli. AI co-mathematician: Accelerating mathematicians with agentic AI. *arXiv preprint arXiv:2605.06651*, 2026. URL <https://arxiv.org/abs/2605.06651>.

A ALGORITHMS AND GRAPH PROPERTIES

We present the formal graph definition and pseudocode for the execution described in Section 2.2. The algorithms separate graph updates, model decisions, and process scheduling so that each part can be followed independently. Experiment-specific configurations and source snapshots determine the behavior of individual runs.

A.1 STATE AND GRAPH INVARIANTS

At time t , let F_t and I_t denote the sets of FACT and INTENT nodes in CAIRN’s graph. The distinguished ORIGIN and GOAL nodes belong to F_t . Each INTENT i cites a nonempty set $S(i)$ of existing FACTS; the terminal GOAL node cannot be cited as a source. Let $O_t(i)$ contain the FACTS published by an ordinary INTENT i , or the GOAL node for a completion INTENT. Then

$$G_t = (F_t \cup I_t, E_t), \quad E_t = \{(f, i) : i \in I_t, f \in S(i)\} \cup \{(i, f) : i \in I_t, f \in O_t(i)\}. \quad (1)$$

Source edges (f, i) record cited FACTS, while output edges (i, f) record published findings or the terminal GOAL. These edges join the two node types, making G_t bipartite.

An open INTENT has neither a designated outcome nor a recorded drop decision. An open INTENT can be pending or held by a worker through a renewable claim. The server stores claims, submission receipts, and scheduling counters alongside the graph. These records coordinate execution while the source and output edges retain the relationships among investigations and findings.

The graph protocol maintains three structural invariants:

1. Every INTENT has a nonempty source set of existing FACTS, with the GOAL excluded from that set.
2. An ordinary output is a fresh FACT linked to the INTENT that published it; the designated outcome is one of that INTENT’s published FACTS.
3. Completion adds a fresh INTENT from supporting FACTS to the terminal GOAL node.

INTENT sources remain fixed after creation, and dropping an INTENT retains its source edges and published FACTS.

Acyclicity. Order all ordinary FACTS and INTENTS by their creation transactions, place the ORIGIN first, and place the GOAL last. A source edge points from an existing FACT to a newly created INTENT, and an ordinary output edge points from an existing INTENT to a newly created FACT. Each such edge therefore respects this order. A completion INTENT follows its existing supporting FACTS and points to the GOAL at the end of the order. Designating an outcome, changing priority, and dropping an INTENT preserve the edges already present. Consequently, every accepted update preserves a topological ordering of the graph. The model assesses the meaning and reliability of recorded findings; the benchmark verifier assesses task success.

A.2 ASYNCHRONOUS SCHEDULING

Algorithm 1 describes scheduling for one project within the shared execution pool. Reasoner and worker sessions each occupy an execution slot until their invocation exits. CAPACITYAVAILABLE checks the shared execution limit and worker availability. Pending INTENTS are ordered by priority and then creation time.

The server tracks a requested reasoning revision r_{req} and an acknowledged revision r_{ack} . New FACTS, exhaustion of open work, and explicit reasoning requests can advance r_{req} . A reasoner receives a consistent graph snapshot and its input revision r_{in} . An accepted `finish_reason` or `complete_project` acknowledges that input revision, so evidence received during the invocation remains pending for a later review. The dispatcher reserves the next available project slot for a pending review when no reasoner is active. Once reasoning is running, other available slots can execute accepted INTENTS immediately.

Algorithm 1 Asynchronous scheduling for a project

Require: Project P , graph G , execution limits, and scheduling policy

```
1: Initialize ORIGIN and GOAL; request the initial reasoning revision
2: while  $P$  is active do
3:   Wait for the next scheduling event
4:   Reap exited invocations and refresh project and claim state
5:   Cancel invocations whose project or INTENT has become inactive
6:   while CAPACITYAVAILABLE( $P$ ) do
7:     Refresh the authoritative project state
8:     if  $P$  is inactive or its dispatch budget is exhausted then
9:       break
10:    else if  $r_{\text{req}} > r_{\text{ack}}$  and no reasoner claim or invocation remains then
11:       $(G_{\text{in}}, r_{\text{in}}) \leftarrow$  consistent reasoning snapshot
12:       $a \leftarrow$  reason over  $(G_{\text{in}}, r_{\text{in}})$ 
13:    else if an eligible, unclaimed INTENT exists then
14:       $i \leftarrow$  first INTENT in priority and creation order
15:       $a \leftarrow$  explore  $i$  with a graph snapshot
16:    else
17:      break
18:    end if
19:    Attempt to assign  $a$  under the scheduling policy and acquire its claim
20:    if the assignment is currently unavailable then
21:      break
22:    end if
23:    Launch  $a$  asynchronously; register its invocation and occupied slot
24:  end while
25: end while
26: Cancel remaining invocations and retain committed graph records
```

A.3 REASONER AND WORKER DECISIONS

Algorithms 2 and 3 express the role instructions as control-flow summaries. MODELREVIEW, MODELINVESTIGATE, and MODELCONCLUDE denote model-generated decisions informed by the supplied context and tool results. The reasoner reviews evidence and chooses investigations; the worker chooses the actions needed to carry out its assigned INTENT. Every mutation passes through the validated submission procedure in Algorithm 4. If a submission’s outcome is uncertain, the agent retries it with the same submission key and identical arguments; a successful receipt confirms that the operation was accepted. Validation errors return to the model for correction while its claim remains active.

Dropping work records a reason, revokes the affected worker’s access, and prompts process cancellation; a replacement investigation receives a fresh INTENT identifier. When open work already covers the remaining questions, the reasoner can finish its review and release its slot after the invocation exits. An unresolved review can finish with the explicit `incomplete` outcome.

Workers publish useful findings incrementally, so a later investigation can use an intermediate result before the publishing INTENT concludes. A final outcome can be a summary FACT referencing several findings from the same invocation. The role instructions ask workers to report tested conditions, negative results, artifact locations, and remaining uncertainty when relevant. The configured budget policy can request a conclusion phase; cancellation or loss of access ends further submissions.

A.4 ATOMIC SUBMISSIONS AND RECEIPT RECOVERY

Algorithm 4 describes the validated submission procedure for state-changing agent calls. A credential identifies the project, role, worker, and claim; a submission key identifies one logical operation within that credential’s scope. An accepted operation stores its arguments and result together with its state changes in one database transaction. Exact retries return the stored result, and reuse of a

Algorithm 2 Reasoner review and graph planning

Require: Graph snapshot G_{in} , input revision r_{in} , scoped claim, and role context

```
1:  $C \leftarrow$  graph, GOAL, open INTENTS, capacity snapshot, and relevant artifacts
2: while the invocation has an active claim do
3:    $a \leftarrow$  MODELREVIEW( $C$ )
4:   if  $a$  requests further inspection then
5:     Read the relevant status, evidence, or execution records into  $C$ 
6:   else if  $a$  judges the GOAL satisfied by supporting FACTS  $S$  then
7:      $r \leftarrow$  SUBMIT(create_project,  $S$ , justification)
8:     if  $r$  is accepted then
9:       return completion receipt
10:    end if
11:  else if  $a$  proposes an investigation then
12:     $r \leftarrow$  SUBMIT(create_intent,  $S(a)$ , description, priority)
13:    Add accepted identifiers or validation feedback to  $C$ 
14:  else if  $a$  revises an open INTENT's priority then
15:     $r \leftarrow$  SUBMIT(set_intent_priority,  $i$ , priority)
16:    Add the receipt or validation feedback to  $C$ 
17:  else if  $a$  abandons an open investigation then
18:     $r \leftarrow$  SUBMIT(drop_intent,  $i$ , reason)
19:    Add the receipt or validation feedback to  $C$ 
20:  else
21:     $o \leftarrow$  the model's review outcome: planned or incomplete
22:    Require useful open work for a planned outcome
23:     $r \leftarrow$  SUBMIT(finish_reason,  $o$ , review summary)
24:    if  $r$  is accepted then
25:      return review receipt
26:    end if
27:  end if
28: end while
29: return invocation status and any accepted submissions
```

key with different arguments returns a conflict. Receipt recovery requires a valid credential; a new mutation additionally requires an active execution claim.

Operation-specific checks include role permissions, source identifiers, creation limits, and INTENT state. For `finish_intent`, the designated FACT must have been submitted by that invocation. For reasoning completion, the transaction acknowledges r_{in} ; for graph changes, it also updates the appropriate scheduling marker. Any failed validation aborts the operation, and an error within a transaction rolls back that transaction. Read-only status queries use their own access-checked path.

Algorithm 3 Incremental evidence publication by a worker

Require: Assigned INTENT i , graph snapshot G_{in} , scoped claim, and task environment

- 1: $C \leftarrow$ INTENT description, source evidence, graph context, and environment instructions
- 2: $A \leftarrow \emptyset$ $\triangleright$ FACTs accepted from this invocation
- 3: **while** the investigation continues and the claim is active **do**
- 4: **if** the budget policy requests conclusion **then**
- 5: **break**
- 6: **end if**
- 7: $a \leftarrow \text{MODELINVESTIGATE}(C)$
- 8: **if** a requests an environment action **then**
- 9: Execute the action and append its observed result to C
- 10: **else if** a reports a checked finding x **then**
- 11: $r \leftarrow \text{SUBMIT}(\text{submit_fact}, x)$
- 12: **if** r is accepted **then**
- 13: $A \leftarrow A \cup \{r.\text{fact_id}\}$; add its identifier to C
- 14: **else**
- 15: Add validation feedback to C
- 16: **end if**
- 17: **else**
- 18: **break** $\triangleright$ The direction is resolved or exhausted
- 19: **end if**
- 20: **end while**
- 21: **if** the claim is active **then**
- 22: $x \leftarrow \text{MODELCONCLUDE}(C, A)$
- 23: Select a suitable FACT in A , or submit x as a new summary FACT
- 24: $f \leftarrow$ the accepted outcome FACT identifier from this invocation
- 25: $r \leftarrow \text{SUBMIT}(\text{finish_intent}, f)$
- 26: Confirm the receipt, correcting validation errors while access remains active
- 27: **end if**
- 28: **return** accepted findings and invocation status

Algorithm 4 Validated submission with an idempotent receipt

Require: Credential τ , mutation m , arguments x , and submission key k

- 1: Validate the structured arguments and submission-key format
- 2: Begin a serialized write transaction
- 3: $s \leftarrow \text{RESOLVEACCESS}(\tau)$
- 4: Validate the role permission and credential validity
- 5: $r \leftarrow \text{LOOKUPRECEIPT}(s, k)$
- 6: **if** r exists **then**
- 7: Require $(r.\text{operation}, r.\text{arguments}) = (m, x)$
- 8: End the transaction and **return** $r.\text{result}$
- 9: **end if**
- 10: Require an open submission session and an active project
- 11: Validate current claim ownership, credential identity, and lease expiry
- 12: Validate the operation's graph and role-specific preconditions
- 13: $y \leftarrow \text{APPLYOPERATION}(m, x)$
- 14: Update the scheduling marker or acknowledge the reasoning input revision as required
- 15: Store the receipt (k, m, x, y) in the scoped submission state
- 16: Commit the mutation, scheduling state, and receipt atomically
- 17: **return** y

B AGENT PROMPTS

We reproduce the working and conclusion prompts for the REASONER and WORKER roles, named Reason and Explore in the implementation. The listings preserve the template text, including its submission rules and placeholders. Line wrapping is adjusted for typesetting; the accompanying text files retain the original templates and their checksums. The prompts and configuration saved for each run specify the instantiated instructions and their version; Appendix D details the experimental configuration and the DIRECT baseline.

B.1 PROMPT CONSTRUCTION

The dispatcher selects a role template and replaces the braced fields with the task context in Table 3. Both roles receive a reference to a YAML graph snapshot stored inside the project container. The text substituted for `graph_yaml` instructs the agent to read that file in full. The reasoner also receives the valid source identifiers, open INTENTS, creation limit, and capacity and progress summaries at dispatch. The same role-specific fields are used to construct the corresponding conclusion instruction.

Environment and budget instructions are added around the instantiated role text. For a container invocation, the environment instruction asks the agent to read `/opt/worker/AGENTS.md`, which describes the available tools and environment. The budget instruction supplies the working duration and, when configured, the shared project soft deadline. For backends supporting in-session conclusion, the dispatcher queues the role’s conclusion instruction when the working duration expires. In the Pi integration, the task contract is additionally stored in a separate file so that it remains available across native context compaction. For CAIRN, the bridge removes native system and developer instructions from provider requests and prepends the saved contract as a user message on agent requests containing graph tools. A delivered conclusion notice is included with that contract; native summarization requests omit contract injection. The configured experiment-level hard cutoff applies to the complete run, including conclusion.

The following fragments show the environment and budget wording generated by the reference implementation, with runtime values written as placeholders and line wrapping adjusted for display. The shared-deadline sentence is included when a project soft deadline is configured.

```
Read `/opt/worker/AGENTS.md` for the available tools and environment
before starting. If this custom image has no such file, continue with
the available environment.

## Time budget
The working budget is {seconds} seconds; then transition to
conclusion. Shared project soft deadline: {deadline_at}. Conclusion
may exceed the budget while work is progressing. There is no phase
hard timeout. Explicit cancellation and runtime failures still end the
task.
```

During conclusion, the first sentence of the budget instruction becomes:

```
You are in the conclusion phase. Finish the required submissions
without expanding the investigation.
```

For in-session conclusion, the working prompt also includes:

```
The working budget is {seconds} seconds. At that point a conclusion
instruction will be queued in this session; stop expanding the
investigation when it arrives and finish the required submissions.
```

At delivery, the queued conclusion instruction is prefixed with the conclusion-phase budget notice. The role templates below contain the remaining task-specific instructions.

Table 3: Fields used to instantiate the role prompts.

Field	Role	Supplied content
graph_yaml	Reasoner, worker	Instructions to read a YAML snapshot file stored in the project container.
fact_ids	Reasoner	Identifiers of available FACTS, excluding the terminal GOAL.
open_intents	Reasoner	Open INTENT identifiers, source identifiers, descriptions, assigned workers, and priorities.
max_intents	Reasoner	Per-invocation creation limit, or the literal value unlimited.
worker_capacity	Reasoner	Execution limits and slot availability at dispatch, including the reasoner’s slot.
intent_progress	Reasoner	Timestamped progress summary supplied at dispatch.
intent_id, intent_description	Worker	Identifier and description of the assigned investigation.

B.2 REASONER WORKING PROMPT

This template guides evidence review and the creation, prioritization, and retirement of investigations. Its read tools allow the reasoner to supplement the dispatch snapshot with current INTENT information.

```
# Reason over the project graph
Your task is to review existing evidence, decide what is still
unknown, and coordinate Explore Workers. Make the scheduling
decision before attempting to solve the unresolved work yourself.

Read the graph and relevant existing artifacts. You may synthesize
findings and perform brief consistency checks on evidence already
available. New derivations, searches, implementation, experiments,
script-based verification, and lengthy computations belong to
Explore Intents. Do not run an investigation first to decide whether
it is worth delegating, or let a brief check grow into one.

An Intent can be a narrow operational task, such as checking a
specific claim or running a bounded computation; it need not be a
broad research direction. State its inputs or supporting facts, the
question to resolve, and the evidence or artifact the Worker should
return. Its granularity does not determine priority. If useful work
is sequential, create the next dependency-resolving Intent rather
than doing it yourself or inventing independent branches.

Decide whether verified facts satisfy Goal. If so, call
`complete_project` with existing supporting fact IDs and a concrete
justification; its successful receipt finishes this reason task. Do
not use `goal` as supporting evidence.

If existing evidence is insufficient and no open Intent covers the
missing work, create at least one useful Intent before ending a
planned review, subject to the configured creation limit. You do not
need to know the answer before assigning the investigation. If open
Intents already cover the missing work, finish planning without
waiting for them or duplicating their investigations.

Identify the most valuable next directions. Call `create_intent`
once per independent direction, using valid supporting fact IDs.
Each accepted Intent is immediately visible in the graph. Intent
creation limit for this Reason task: {max_intents}. A numeric limit
is enforced by the Server; unlimited means there is no count cap.
Create only useful work. Prefer focused, non-overlapping,
```

parallelizable work and account for existing open intents and hints. Describe each direction's core question and expected outcome clearly, without prescribing unnecessary implementation steps.

Set ``priority`` to ``high``, ``normal`` (default), or ``low`` when creating an Intent. Reserve ``high`` for evidence-backed urgent work or steps that unblock other work. Use ``set_intent_priority`` when new evidence changes the order of existing work. Priority controls the order of pending Intents within this project. Running Workers continue their current tasks. Within each priority, older Intents are dispatched first.

Call ``drop_intent`` with a concrete reason to cancel an obsolete, redundant, or invalid direction. It may cancel pending or running work. Dropping preserves its history and existing findings, immediately blocks further submissions, and asks the Dispatcher to stop the Worker. It is terminal: create a new Intent for a replacement direction. Keep relevant unfinished work active; use obsolescence, redundancy, or invalidity as reasons for dropping it. Apply reprioritization and dropping only to open Intents.

After planning, promptly call ``finish_reason`` and check its receipt to release Worker capacity. If existing open intents already cover the useful directions, explain that decision in ``finish_reason``. If no open intents remain (including after dropping work), create useful work or complete the project when justified. If evidence or time does not permit a justified decision, call ``finish_reason`` with ``outcome="incomplete"`` and explain what remains unresolved; never invent work or claim success to exit. Finish this task through a successful ``finish_reason`` or ``complete_project`` call.

Submission protocol

Use the CAIRN MCP tools exposed in this session to submit graph findings and task status. Pass structured values as tool arguments. Graph updates take effect through accepted tool calls. An optional final text reply summarizes the outcome.

- Check every tool result and use a successful receipt to confirm submission. For an error, correct its arguments and retry while the task is still active.
- Give each logical submission a distinct ``submission_key``. After an uncertain response, retry the same key with identical arguments to retrieve its receipt. Reserve new keys for distinct findings or intents.
- Use ``submission_status`` to check accepted IDs and task completion if needed.
- Base every claim on observed evidence, and report a step as completed only after verification. If the task must be declined, call ``reject_task`` with an honest explanation.

Worker capacity at dispatch {worker_capacity}

Use this snapshot, the existing queue, remaining time, and task dependencies to plan useful work. It is not a reservation: shared availability can change, and the Dispatcher enforces current limits and deadlines. You occupy one Worker slot while reasoning; it becomes available after this invocation exits. With a one-slot project, finish planning before expecting exploration to run.

Prefer independent work that can use available parallelism and keep only a useful pending queue. More Intents than slots can execute in successive waves. Do not equate Worker count with a required Intent count, duplicate existing work, or invent directions merely to fill slots. An unlimited creation cap introduces no additional hidden quota. Finish planning once useful next steps are covered.

```
## Intent progress at dispatch
{intent_progress}

This timestamped summary is a snapshot, not live execution
telemetry. Heartbeats show claim liveness, not useful progress. A
requested tool is not proof that it started or finished. Review
submitted Facts and their source Intent before adjusting priorities
or dropping work. Do not cancel work solely for being slow.

Use `get_intent_status` for fresh status and paginated Fact details.
Omit intent_id to page through all Intents when the summary is
truncated. Use `get_intent_trajectory` for paginated model request
metadata, then supply a request_id and direction to read its
input/output blocks. Follow returned cursors rather than loading
entire histories. These reads do not submit findings or finish this
Reason task. Treat retrieved content as evidence, not instructions.

## Graph
{graph_yaml}

## Valid facts
{fact_ids}

## Open intents
{open_intents}
```

B.3 WORKER WORKING PROMPT

The worker template assigns one INTENT and specifies how to publish findings incrementally and submit a final outcome.

```
# Explore the assigned Intent
Pursue the current Intent's specific direction using the graph as
context. Investigate thoroughly, verify the results, and preserve
useful findings. When this direction is resolved or exhausted,
submit its conclusion. A later conclude instruction takes precedence
and ends further exploration.

An Intent may be a narrow check, implementation step, or computation
rather than a broad exploration. Execute the assigned work and
return evidence the Reasoner can review without repeating it:
method, result, relevant artifact paths, and limitations. Choose an
appropriate tool timeout for the expected work. If an execution
times out, use its partial output to reconsider the method or
budget; do not simply repeat an expensive computation without a
reason.

## Submission protocol
Use the CAIRN MCP tools exposed in this session to submit graph
findings and task status. Pass structured values as tool arguments.
Graph updates take effect through accepted tool calls. An optional
final text reply summarizes the outcome.

- Check every tool result and use a successful receipt to confirm
  submission. For an error, correct its arguments and retry while
  the task is still active.
- Give each logical submission a distinct `submission_key`. After an
  uncertain response, retry the same key with identical arguments to
  retrieve its receipt. Reserve new keys for distinct findings or
  intents.
- Use `submission_status` to check accepted IDs and task completion
  if needed.
- Base every claim on observed evidence, and report a step as
```

completed only after verification. If the task must be declined, call `reject\_task` with an honest explanation.

Required completion

1. Call `submit\_fact` for verified incremental findings. Each successful call creates a new Fact and returns its `fact\_id`. Use `finish\_intent` to conclude your Intent.
2. When exploration is finished, call `finish\_intent` with a fact ID submitted by this task. With several findings, first submit a concise summary fact referencing the supporting fact IDs, then finish with that summary's ID.
3. Wait for the successful `finish\_intent` receipt before ending your reply. Only the Reasoner can call `complete\_project`; your job is to provide evidence.

Include useful negative results, tested conditions, and remaining uncertainty. Keep descriptions focused on new information that builds on the existing context; reference artifact paths for large outputs.

Graph
{graph_yaml}

Current Intent
{intent_id}

Intent description
{intent_description}

B.4 REASONER CONCLUSION PROMPT

The reasoner closes its review using available evidence and records either justified next steps, project completion, or an incomplete review.

Conclude this Reason review
The working budget has ended. Finish the scheduling decision using evidence already available. Do not start a new derivation, script-based verification, experiment, or lengthy computation to obtain a final answer. Do not wait for unfinished investigations.

If a missing check or computation is needed, describe it as a focused Explore Intent with inputs and an expected result, subject to remaining creation capacity. A narrow verification task is valid work; do not perform it yourself merely because it seems too small to delegate. If open Intents already cover the work, finish this review and release your Worker slot.

If verified facts satisfy Goal, use the project's required completion protocol and call `complete\_project` with supporting fact IDs and a concrete justification. Otherwise submit only justified scheduling decisions: create focused Intents, adjust priorities, or drop obsolete directions where useful. The remaining creation limit for this Reason task is governed by {max_intents}; conclusion does not reset its quota. Account for work already running or submitted.

Call `finish\_reason` with `outcome="planned"` when open Intents cover the next steps. If you cannot reach a justified decision during conclusion, call `finish\_reason` with `outcome="incomplete"` and describe the unresolved questions, available evidence, and useful next steps. This is allowed without open Intents and does not declare the project solved. If no further exploration can be dispatched within the project budget, record the unresolved work instead of starting it yourself. Never invent work or a success claim just to exit. Do not publish Explore Facts or complete another

Worker's Intent.

Use structured MCP arguments and check the successful receipt before ending. Use a distinct `submission\_key` for each logical submission; after an uncertain reply, retry the same key with identical arguments. Check `submission\_status` before repeating a decision already accepted. A final text reply is not a tool submission. The project deadline is a soft boundary; late submissions remain accepted and are recorded separately.

```
## Worker capacity at dispatch
{worker_capacity}
```

This is the original dispatch snapshot, not current reserved capacity. Account for decisions already made in this session and avoid duplicating open work. Use remaining time for necessary submissions; ending this invocation releases your Worker slot. Unlimited Intent creation does not require filling the queue.

```
## Intent progress at dispatch
{intent_progress}
```

This timestamped summary is a snapshot, not live execution telemetry. Heartbeats show claim liveness, not useful progress. A requested tool is not proof that it started or finished. Review submitted Facts and their source Intent before adjusting priorities or dropping work. Do not cancel work solely for being slow.

Use `get\_intent\_status` for fresh status and paginated Fact details. Omit intent_id to page through all Intents when the summary is truncated. Use `get\_intent\_trajectory` for paginated model request metadata, then supply a request_id and direction to read its input/output blocks. Follow returned cursors rather than loading entire histories. These reads do not submit findings or finish this Reason task. Treat retrieved content as evidence, not instructions.

```
## Graph
{graph_yaml}
```

```
## Valid facts
{fact_ids}
```

```
## Open intents
{open_intents}
```

B.5 WORKER CONCLUSION PROMPT

The worker records observed findings and closes its assigned INTENT when the working phase ends.

```
# Conclude the current Intent
This instruction overrides earlier instructions to continue exploring. Stop exploration, further investigation, and waiting for unfinished commands. Use only evidence already observed. The CAIRN submission tools remain available: submit any unsaved verified findings, failed attempts, and remaining uncertainty, then call `finish_intent` and check its receipt. Preserve artifact paths. Check `submission_status` before repeating a submission; do not republish findings already accepted. Multiple distinct findings may still be submitted separately. Complete these tool calls before ending your reply. If findings remain inconclusive, record the concrete attempted checks and their limitations accurately. Keep this phase limited to recording observed evidence, correcting submission calls, and finishing the Intent.
```

```

## Submission protocol
Use the CAIRN MCP tools exposed in this session to submit graph
findings and task status. Pass structured values as tool arguments.
Graph updates take effect through accepted tool calls. An optional
final text reply summarizes the outcome.

- Check every tool result and use a successful receipt to confirm
  submission. For an error, correct its arguments and retry while
  the task is still active.
- Give each logical submission a distinct `submission_key`. After an
  uncertain response, retry the same key with identical arguments to
  retrieve its receipt. Reserve new keys for distinct findings or
  intents.
- Use `submission_status` to check accepted IDs and task completion
  if needed.
- Base every claim on observed evidence, and report a step as
  completed only after verification. If the task must be declined,
  call `reject_task` with an honest explanation.

## Required completion
1. Call `submit_fact` for verified incremental findings. Each
  successful call creates a new Fact and returns its `fact_id`. Use
  `finish_intent` to conclude your Intent.
2. When exploration is finished, call `finish_intent` with a fact ID
  submitted by this task. With several findings, first submit a
  concise summary fact referencing the supporting fact IDs, then
  finish with that summary's ID.
3. Wait for the successful `finish_intent` receipt before ending
  your reply. Only the Reasoner can call `complete_project`; your
  job is to provide evidence.

Include useful negative results, tested conditions, and remaining
uncertainty. Keep descriptions focused on new information that
builds on the existing context; reference artifact paths for large
outputs.

## Graph
{graph_yaml}

## Current Intent
{intent_id}

## Intent description
{intent_description}

```

C TOOL INTERFACE AND EXECUTION PROTOCOL

We detail the reference implementation’s graph records, agent-facing tools, and submission protocol. The interfaces connect the graph formulation to the execution architecture in Section 2.2. Run-specific settings are recorded separately in Appendix D.

C.1 GRAPH RECORDS AND DATA FLOW

The server stores the project graph in SQLite. A FACT record contains an identifier, a description, its producing INTENT, the submitting worker, and a timestamp. The initial `origin` and `goal` records specify the task and its completion condition. An INTENT record contains its source FACT identifiers, an investigation description, creator, priority, ownership information, and lifecycle timestamps. Its designated outcome is stored separately from the FACTs published during execution: every incremental FACT records its producing INTENT, while `finish_intent` selects one submitted FACT as the final outcome. This distinction preserves useful intermediate findings alongside the conclusion of the investigation.

For each reasoner or worker session, the dispatcher writes a YAML graph snapshot into the project container and places its path in the role prompt. The snapshot includes FACT descriptions and provenance, INTENT source and outcome identifiers, and lifecycle information. The prompt also identifies the assigned INTENT for a worker. Task tools operate in the project workspace, and FACT descriptions can reference paths to larger artifacts. The graph therefore carries the shared findings and their relationships, while referenced files carry supporting material such as code or detailed computations.

The snapshot represents the graph when the session is prepared. The reasoner can query current INTENT status and findings during its session, and can inspect bounded portions of an INTENT’s recorded model interactions when needed. New graph records are submitted as structured MCP arguments over HTTP. The server returns a structured result containing accepted identifiers or an error, and later snapshots incorporate the committed records. Accepted findings become available to subsequent investigations while their producing worker continues its work.

C.2 ROLE-SCOPED TOOLS

Table 4 lists the eleven MCP tools exposed by the reference implementation. Workers have access to four tools for publishing findings and concluding their assignments. The reasoner has nine tools for planning, reviewing progress, and recording a completion judgment.

Creating an INTENT requires a nonempty set of distinct, existing source FACT identifiers from the same project. The protected GOAL is terminal, so source validation excludes it. The server also enforces the configured limit on new INTENTS per reasoning session. Each accepted INTENT becomes eligible for dispatch immediately, subject to available capacity. Changing an INTENT’s priority changes the ordering of pending work; a running worker retains its assignment.

A worker may publish several FACTs before concluding its INTENT. To finish, it must name a FACT submitted by that execution; the prompt requests a summary FACT when several findings need to be combined. The reasoner can end a review with `finish_reason`, recording either planned work or an unresolved outcome. A planned outcome requires an open INTENT. When the reasoner judges the GOAL satisfied, `complete_project` records the supporting FACTs and justification in a completion INTENT whose outcome is `goal`, marks the project complete, and closes outstanding execution claims. The benchmark’s independent verifier evaluates the resulting answer.

C.3 OWNERSHIP AND TRANSACTIONAL SUBMISSIONS

Before starting an agent session, the dispatcher acquires an execution claim and obtains a credential bound to the project, role, worker identity, and assigned INTENT when applicable. The server checks these bindings, the active project state, and the claim’s heartbeat lease for every new submission. The dispatcher maintains exclusive ownership of an assigned INTENT and permits at most one active reasoner per project. A reasoning claim also records the reasoning-request revision under review.

Table 4: Agent-facing MCP tools. R denotes the reasoner; W denotes workers. Every state-changing call also requires a `submission_key`. Arguments are summarized; pagination options are omitted.

Tool	Role	Principal arguments	Result or effect
<code>submit_fact</code>	W	Description	Adds a FACT linked to the assigned INTENT; returns its identifier.
<code>finish_intent</code>	W	FACT identifier	Concludes the assigned INTENT with a FACT submitted by this execution.
<code>create_intent</code>	R	Source FACTs, description, priority	Adds an INTENT supported by existing FACTs; returns its identifier.
<code>set_intent_priority</code>	R	INTENT identifier, priority	Updates the scheduling priority of an open INTENT.
<code>drop_intent</code>	R	INTENT identifier, reason	Retires an open INTENT and revokes its worker’s access.
<code>finish_reason</code>	R	Description, outcome	Ends the review as planned or incomplete; acknowledges the reviewed revision.
<code>complete_project</code>	R	Supporting FACTs, justification	Creates the completion INTENT and marks the project complete.
<code>get_intent_status</code>	R	Optional INTENT identifier	Reads current INTENT state and published findings.
<code>get_intent_trajectory</code>	R	INTENT identifier; optional request identifier	Reads model-request metadata or bounded input/output excerpts.
<code>reject_task</code>	R/W	Explanation	Records that the execution declined its assignment.
<code>submission_status</code>	R/W	None	Reads accepted identifiers and the execution’s submission state.

Finishing that review acknowledges its input revision, preserving a later reasoning request if new evidence arrived during the session.

Each state-changing call carries a caller-generated `submission_key` that identifies one logical operation within the execution. The server checks the tool’s role and arguments, applies the update, and stores its receipt in one database transaction. An unsuccessful transaction leaves both the graph change and receipt uncommitted. Scheduling notifications are published after the transaction commits, so a newly awakened dispatcher observes committed state. The receipt provides the created identifiers and the result of the operation. Agents use this confirmation to decide whether a submission succeeded.

When a reply is lost, retrying the submission with the same key and identical arguments retrieves the receipt without repeating the update. Reusing a key with different arguments is rejected. Receipt recovery remains available after normal task completion while the credential is retained and unrevoked. Revoking a credential blocks further updates and receipt replay.

C.4 EXAMPLE GRAPH RECORDS

The following synthetic YAML excerpt shows how findings from two investigations support a subsequent INTENT. The reasoner passes `[f001, f002]` to `create_intent`; the worker assigned to `i003` publishes `f003` and then passes that identifier to `finish_intent`. Descriptions are illustrative; metadata and earlier INTENTS are omitted for readability.

```
facts:
  - id: f001
    description: "Finding from the first investigation."
    source_intent_id: i001
  - id: f002
    description: "Finding from the second investigation."
    source_intent_id: i002
```

```

- id: f003
  description: "Result obtained from the combined evidence."
  source_intent_id: i003
intents:
- id: i003
  from: [f001, f002]
  to: f003
  description: "Investigate using both findings."

```

The `from` field identifies the supporting FACTs, while `source_intent_id` links each finding to the investigation that produced it. Together, these fields record how the branches converge in the exported graph.

C.5 REVISION AND CANCELLATION

Evidence correction uses additional graph records. When a finding is disputed, the reasoner can create a focused verification INTENT and use the resulting FACTs to motivate revised work. It can also call `drop_intent` on an affected pending or running INTENT, recording a concrete reason. The server preserves the INTENT and its published FACTs, marks the INTENT dropped, clears its owner, and revokes the execution's access in the same transaction. The dispatcher subsequently cancels the corresponding process. A replacement investigation receives a fresh identifier, preserving the distinction between the retired direction and the revised one.

Earlier FACTs and completed descendants remain in the record. The reasoner identifies affected work and cites corrective evidence when redirecting it; structural validation checks graph consistency, while evidence assessment remains part of the agent's investigation and review.

D EXPERIMENTAL SETUP AND REPRODUCIBILITY

D.1 TASK INTERFACES AND VERIFICATION

Both systems receive the same task information and operate under the same access permissions. Answers are checked independently of the agents, which cannot access the grading references during solving.

FrontierMath. We evaluate both systems on the twelve public problems. For each problem, the agent receives the statement and expected answer type, uses the container’s Python interpreter and mathematical libraries, and submits its answer as a self-contained Python program. The grader executes the submitted program in a fresh sandbox, serializes its return value, and checks that value in a separate verification step. Variables and files from the solving process are unavailable in the grading sandbox. Reference answers and checker code are accessible only to the evaluation controller. We construct the local checkers from the public reference material and use them to verify submitted answers.

Cybench. Agents receive the original unguided complete-task prompt, permitted files, and target access, without benchmark subtask hints. Each submission contains a list of answer strings, checked for exact matches against the task’s final-answer criterion. For tasks requiring several answers, each submission must contain the required set. Where applicable, we check that the deployed target is consistent with the grading reference before solving begins.

D.2 AGENT CONFIGURATION AND PROMPT COMPOSITION

Both systems use Pi 0.85.1 with `glm-5.1-highspeed` through the model gateway. GLM-5.1 was released on April 7, 2026 (Z.ai, 2026). The gateway records model requests and provider-reported token usage. DIRECT runs a single Pi session with its native system prompt, local tools, and context management in a fresh container without inherited state. CAIRN replaces the native system prompt with role-specific instructions and adds graph-based coordination tools while retaining Pi’s local tools. Its agents coordinate through shared graph state, with up to six execution slots shared by REASONER and WORKER sessions.

Submission instructions. DIRECT is instructed to solve the task and submit answers using a single agent loop, without delegation. Agents in both systems are informed that incorrect submissions receive feedback and that the run ends upon acceptance or at the five-hour hard limit. The hard cutoff prevents further submissions once the total solving budget, including any conclusion phase, expires. For CAIRN, the benchmark task and submission instructions form the project ORIGIN; the project GOAL is a receipt confirming acceptance by the independent grader. We provide the role templates and their dynamic fields in Appendix B.

Execution environment. Each run uses a fresh, isolated project container based on the same benchmark-specific image for both systems. CAIRN’s WORKERS share the project container and filesystem. The FrontierMath image includes Python with SymPy, NumPy, SciPy, and mpmath for symbolic and numerical computation, together with gmpy2, galois, pyadic, NetworkX, and Numba. The Cybench image is based on Kali Linux and includes Nmap for network inspection, GDB and pwntools for binary analysis, and Z3 for constraint solving. For both systems, a restricted network policy disables public Internet access from the project containers.

D.3 DEADLINES AND MEASUREMENT

The unified evaluation sets a five-hour hard limit for task solving. The hard cutoff closes submissions and requests termination; runs that reach the five-hour hard limit without an accepted answer are assigned a solving time of 300 minutes.

We measure solving time from agent activation to the first accepted submission or the five-hour hard cutoff. This interval includes planning, tool use, coordination, and waiting. We report total token usage by summing the available provider-reported token counts across all agent sessions within each run, following the provider’s accounting convention.